

Centro de Estudios Superiores del Noroeste

Ing. Desarrollo de Software



Revisión de Arquitecturas de Software

Fabian Camacho Hernandez

Arquitectura de Software

FASE 1.

Métodos HTTP Semánticos.

Los métodos HTTP son verbos utilizados en las APIs REST para indicar la acción que se desea realizar sobre un recurso.

- GET: Se utiliza para obtener información de un recurso existente sin modificarlo.

Ejemplo: GET /pacientes/15

Resultado: Devuelve los datos del paciente con ID 15.

Características:

- No modifica datos.
- Puede ejecutarse múltiples veces sin efectos secundarios.
- Es cacheable.

- POST: Se utiliza para crear un nuevo recurso en el servidor.

Ejemplo: POST /pacientes.

Resultado: Se crea un nuevo paciente.

Características:

- Modifica el estado del sistema.
- Generalmente genera un nuevo identificador.

- PUT: Se utiliza para actualizar completamente un recurso existente.

Ejemplo: PUT /pacientes/15.

Características:

- Reemplaza completamente el recurso.
- Es idempotente (si se ejecuta varias veces produce el mismo resultado).

- DELETE: Se utiliza para eliminar un recurso.

Ejemplo: DELETE /pacientes/15.

Características:

- Elimina el recurso solicitado.
- También es idempotente.

Estructura JSON.

JavaScript Object Notation, es un formato ligero de intercambio de datos basado en texto.

Ejemplo:

```
</> JSON

{
  "id": 1,
  "nombre": "Ana López",
  "correo": "ana@email.com",
  "activo": true
}
```

¿Por qué se convirtió en estándar frente a XML?

- JSON es más compacto.

JSON	XML
<pre></> JSON { "nombre": "Ana" }</pre>	<pre></> XML <persona> <nombre>Ana</nombre> </persona></pre>

- JSON es más fácil de leer ya que tiene menos etiquetas y una estructura más sencilla.
- Mejor rendimiento: Menor tamaño de transmisión, menor consumo de ancho de banda y parseo más rápido por aplicaciones web y móviles.
- Compatibilidad nativa con JavaScript, los navegadores pueden convertir JSON directamente en objetos.

Códigos de Error HTTP.

Los códigos HTTP se agrupan en familias:

Familia	Significado
1xx	Informativos
2xx	Éxito
3xx	Redirecciones
4xx	Error del cliente (Front - End)
5xx	Error del servidor (Back - End)

- Errores 4xx: Indican que la solicitud enviada por el cliente tiene algún problema.
 - 400 Bad Request.
 - 401 Unauthorized.
 - 403 Forbidden.
 - 404 Not Found.
- Errores 5xx: Indican que el servidor fallo al procesar una solicitud válida.
 - 500 Internal Server Error.
 - 502 Bad Gateway.
 - 503 Service Unavailable.
- Diferencia entre 400, 404 y 503.

400	404	503
400 Bad Request: La petición tiene datos inválidos o mal formados.	El recurso solicitado no existe.	El servidor está temporalmente fuera de servicio.
<pre></> JSON { "edad": "abc" }</pre> Cuando el sistema esperaba un número.	<pre></> http GET /pacientes/9999</pre> Si ese paciente no existe.	• Base de datos caída. • Servidor en mantenimiento. • Sobrecarga de tráfico. La petición es correcta, pero el servidor no puede atenderla temporalmente.

Especificación OpenAPI (Swagger).

OpenAPI es un estándar para describir y documentar APIs REST.

Swagger es el conjunto de herramientas más popular para trabajar con OpenAPI.

- Objetivo principal.

Permitir que cualquier desarrollador entienda una API sin revisar el código fuente. Funciona como los planos de una construcción.

- Información que documenta.

- Endpoints.
- Métodos HTTP.
- Parámetros.
- Respuestas.
- Códigos de error.
- Modelos de datos.
- Seguridad.

- Ejemplo:

```
</> YAML

/api/pacientes:
  get:
    summary: Obtener pacientes
```

- Beneficios:

- Comunicación entre equipos: Frontend y Backend pueden trabajar en paralelo.
- Generación automática: Permite generar documentación web, clientes SDK y código servidor.
- Pruebas rápidas: Swagger UI permite probar endpoints desde el navegador.

Contratos de Interfaz (Código).

Una interfaz define un conjunto de métodos que una clase debe implementar. No indica cómo se realiza las operaciones, solamente que operaciones existen.

- Ejemplo de Java.

```
</> Java

public interface PacienteRepository {

    Paciente buscarPorId(int id);

    void guardar(Paciente paciente);

    void eliminar(int id);

}
```

- Implementación.

```
</> Java

public class PacienteRepositorySQL
    implements PacienteRepository {

    public Paciente buscarPorId(int id){
        // consulta SQL
    }

}
```

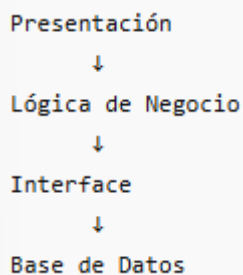
- ¿Por qué se le llama contrato?

Porque establece reglas obligatorias: Si una clase implementa la interfaz, debe cumplir exactamente con los métodos definidos.

- ¿Por qué se considera un contrato inmutable?

Porque las capas del sistema dependen de la interfaz y no de la implementación.

Arquitectura típica:



La lógica de negocio puede seguir funcionando aunque cambie la tecnología de persistencia:

- MySQL.
- PostgreSQL.
- SQL Server.
- MongoDB.

Manejo de Excepciones y Resiliencia.

Una excepción es un evento inesperado que ocurre durante la ejecución de una aplicación.

Ejemplo:

- División entre cero.
- Archivo inexistente.
- Conexión fallida a la base de datos.

- ¿Qué es un Stacktrace?

Es el registro detallado de llamadas que llevaron a un error.

Ejemplo:

```
java.sql.SQLException:  
Connection refused  
  
at Repository.obtenerPaciente()  
at Service.buscarPaciente()  
at Controller.getPaciente()
```

Muestra:

- Tipo de error.
 - Mensaje.
 - Línea donde ocurrió.
 - Secuencia de llamadas.
- ¿Por qué nunca debe mostrarse al usuario?
- Seguridad ya que puede revelar direcciones IP, consultas SQL, Estructura interna del sistema y rutas de archivos.
Esto facilita ataques informáticos.
 - Mala experiencia de usuario: Un usuario final no entiende mensajes técnicos como: "NullPointerException"
 - Buenas prácticas, registrar el error internamente y mostrar un mensaje amigable.

Conclusión (Investigación Fase 1).

Los métodos HTTP, JSON, los códigos de estado HTTP, OpenAPI, los contratos de interfaz y el manejo adecuado de excepciones constituyen pilares fundamentales del desarrollo moderno de APIs y sistemas distribuidos. Su correcta aplicación permite construir aplicaciones más mantenibles, seguras, escalables y fáciles de integrar, facilitando la colaboración entre equipos de desarrollo y mejorando la experiencia de los usuarios finales.

FASE 2.

El sistema de gestión y registro de citas psicológicas se parece más al escenario distribuido por HTTP, ya que su arquitectura está compuesta por contenedores independientes que se comunican mediante protocolos de red. En nuestro diagrama C4 de Nivel 2 identificamos un Frontend desarrollado en React, un Backend API construido con Java y Spring Boot, una Base de Datos MySQL y un servicio externo de correo o SMS para el envío de notificaciones. La comunicación entre estos elementos se realiza mediante solicitudes HTTPS/JSON y consultas a la base de datos utilizando JDBC/SQL, lo cual corresponde al modelo distribuido presentado en el ejemplo del cajero web.

Además, este enfoque resulta más adecuado para el proyecto porque favorece la escalabilidad, el mantenimiento y la disponibilidad del sistema. Si alguno de los componentes presenta fallos, como una pérdida temporal de conexión con la base de datos, el Backend puede gestionar las excepciones y enviar mensajes claros al Frontend para informar al usuario sin provocar la caída total de la aplicación. Esta separación de responsabilidades facilita futuras mejoras, como incorporar aplicaciones móviles, integrar nuevos servicios externos o aumentar la capacidad del sistema a medida que crezca el número de pacientes y psicólogos que utilicen la plataforma.

Conclusión (Fase 2).

El proyecto se asemeja al Escenario 1 (Distribuido por HTTP) porque utiliza una arquitectura basada en contenedores independientes que intercambian información mediante APIs y protocolos de comunicación, alineándose con el modelo C4 que hemos definido para el sistema de citas psicológicas.

Penland, J. (2020, febrero 25). Una guía completa y una lista de códigos de estado HTTP. Kinsta®; Kinsta.
<https://kinsta.com/es/blog/codigos-de-estado-de-http/>

OpenAPI Specification. (s/f). Swagger.io. Recuperado el 7 de junio de 2026, de <https://swagger.io/specification/>

Métodos de petición HTTP. (s/f). MDN Web Docs. Recuperado el 7 de junio de 2026, de <https://developer.mozilla.org/es/docs/Web/HTTP/Reference/Methods>

Manejo de excepciones. (s/f). Academia-lab.com. Recuperado el 7 de junio de 2026, de <https://academia-lab.com/enciclopedia/manejo-de-excepciones/>

IBM Sterling Order Management System. (2026, enero 2). Ibm.com.
<https://www.ibm.com/docs/es/order-management?topic=authentication-interface-contracts-web-ui-framework>

Byte, L. (2024, julio 22). JSON: ¿Qué es este Formato y Cómo Trabajar con Él? CódigoNautas.
<https://codigonautas.com/json-que-es-como-trabajar/>